

MỤC LỤC

Cẩm nang tránh lỗi khi làm Capstone Project ngành Kỹ thuật phần mềm	2
0. Đọc nhanh: 6 nguyên nhân khiến nhóm KHÔNG ĐẠT	2
1. Nhóm lỗi TÀI LIỆU (Final Report)	2
2. Nhóm lỗi SƠ ĐỒ & MÔ HÌNH HÓA	5
3. Nhóm lỗi PHÂN TÍCH NGHIỆP VỤ	7
4. Nhóm lỗi HARDCODE & CẤU HÌNH	9
5. Nhóm lỗi liên quan đến AI trong đề tài	10
6. Nhóm lỗi KIỂM THỬ (Test)	11
7. Nhóm lỗi khi DEMO	11
8. Nhóm lỗi khi TRẢ LỜI CÂU HỎI	13
9. Nhóm lỗi QUY TRÌNH & LÀM VIỆC NHÓM	14
10. Nhóm lỗi về DỮ LIỆU, BẢO MẬT & PHÁP LÝ	14
11. Nhóm lỗi về UI/UX	15
12. CHECKLIST TỰ KIỂM TRƯỚC BẢO VỆ	15
13. NHỮNG CÂU HỎI HAY GẶP NHẤT	17
14. BẢNG TỰ ĐÁNH GIÁ RỦI RO	18
15. Ba lời khuyên quan trọng nhất	18

Cẩm nang tránh lỗi khi làm Capstone Project ngành Kỹ thuật phần mềm

Tài liệu tham khảo dành cho sinh viên chuẩn bị bảo vệ khóa luận tốt nghiệp

Tài liệu này liệt kê những lỗi mà sinh viên thường mắc phải khi làm và bảo vệ đồ án Capstone, kèm cách phòng tránh và checklist tự kiểm. Mục tiêu: giúp nhóm nhìn thấy trước "bẫy" và chuẩn bị đúng chỗ, thay vì phát hiện ra khi đã quá muộn.

0. Đọc nhanh: 6 nguyên nhân khiến nhóm KHÔNG ĐẠT

Hầu hết các trường hợp không đạt hoặc phải bảo vệ lần 2 đều rơi vào 6 nguyên nhân sau:

#	Nguyên nhân	Mức độ phổ biến
1	Demo lỗi ở mainflow, hoặc không demo được hết luồng chính trong thời gian quy định	Rất cao
2	Hardcode các tham số nghiệp vụ (phí, %, ngưỡng, cấu hình)	Rất cao
3	Tài liệu sai/thiếu, không dùng template chính thức của trường	Rất cao
4	Nghiệp vụ không sát thực tế, thiếu ràng buộc ở các tình huống ngoại lệ	Cao
5	Không tiếp thu góp ý của các lần Review đánh giá	Cao
6	Không được GVHD duyệt ra bảo vệ / vắng mặt / thành viên đóng góp quá ít	Trung bình

Một điểm rất đáng chú ý: phần lớn nhóm rơi vào kết quả "đạt nhưng phải chỉnh sửa và nộp lại tài liệu" — nghĩa là **sản phẩm chạy được nhưng tài liệu kéo điểm xuống**. Đây là phần dễ sửa nhất nhưng lại bị bỏ bê nhiều nhất.

1. Nhóm lỗi TÀI LIỆU (Final Report)

Đây là nhóm lỗi bị nhắc nhở nhiều nhất.

1.1. Không dùng template chính thức của trường

Biểu hiện: Tài liệu nhóm nộp khác hẳn với template chính thức trên FLM — sai cấu trúc mục, thiếu mục, thêm mục không có trong mẫu. Hậu quả nặng nhất là phải làm lại và nộp lại toàn bộ tài liệu.

Cách tránh:

- Tải template mới nhất từ FLM **ngay tuần đầu**, không dùng lại file của khóa trước.
- Đối chiếu từng mục trong template với tài liệu của nhóm trước khi nộp.
- Không copy nguyên văn phần hướng dẫn/placeholder trong template rồi để nguyên trong bài nộp.

1.2. Thiếu các trang bắt buộc

Thường bị thiếu nhất:

- Trang **Acknowledgement** — và nếu có thì **quên ghi tên GVHD cùng các thầy cô**, đây là lỗi bị nhắc riêng
- **User Guide / User Manual**
- **Số trang**
- **Danh mục bảng (List of Tables) và danh mục hình (List of Figures)** — phải tách riêng khỏi mục lục chính và đánh lại số thứ tự
- **Bảng từ viết tắt** — thiếu, hoặc có nhưng chưa sắp xếp theo thứ tự
- **Class Diagram**
- **Test case / Testing document** (làm nhưng không nộp)
- Thông tin **mentor, thành viên nhóm**

Cách tránh: Lập bảng kiểm danh mục tài liệu, tick từng mục theo template trước hạn nộp 3 ngày.

1.3. Trình bày cấu trúc

Các lỗi thường gặp:

- Trang bìa canh lề lệch, font chữ lạ, trình bày chưa chỉnh chu
- Trang bìa bị **tràn dòng cuối (địa danh, năm) sang trang sau**
- Mục lục "lố" qua trang bìa; mục lục lặp lại nội dung quá nhiều
- **Không phân trang** cho các phần lớn — Acknowledgement, phần I, II, III... mỗi phần phải bắt đầu ở trang mới
- Giật cấp heading sai (heading level không nhất quán)
- Đánh số trang và số thứ tự mục sai, thụt lề không đồng nhất
- Còn nhiều khoảng trắng thừa
- Định dạng bảng/menu cấu trúc
- **Chèn sơ đồ ở kích thước quá nhỏ, không đọc được chữ bên trong**
- **Chưa xóa các phần gợi ý / tag của template** để nguyên trong bài nộp
- Còn sót tiếng Việt trong tài liệu tiếng Anh (đặc biệt ở phần Test Report)
- Còn sót nội dung không liên quan tới đề tài trong mục Definition/Glossary
- Slide sai thuật ngữ tiếng Anh
- Ghi sai vai trò của GVHD
- **Bảng thuật ngữ (Glossary/Definition) thiếu và chọn lọc không chính xác** — liệt kê từ không cần giải thích, bỏ sót thuật ngữ chuyên ngành quan trọng
- **Nguồn tham khảo không phù hợp, không rõ ràng, thông tin đã lỗi thời** — trích từ blog/nguồn không kiểm chứng, không ghi ngày truy cập

Cách tránh: Dành trọn một buổi cuối cùng chỉ để **đọc lại toàn bộ tài liệu ở chế độ in thử** (print preview). Lỗi trình bày chỉ lộ ra khi nhìn tổng thể trang.

1.4. Viết tài liệu bằng AI nhưng không kiểm tra lại

Đây là lỗi đang tăng mạnh. Biểu hiện:

- Business Rule viết bằng AI nhưng nội dung không chính xác

- Test Report sinh tự động, nhiều thông tin không phù hợp với hệ thống thật
- Non-functional requirements ghi rất nhiều nhưng **không rõ có hiện thực hay không**
- Văn phong chung chung, không gắn với đặc thù đề tài

Cách tránh:

- AI có thể giúp diễn đạt, **không** thay bạn quyết định nội dung.
- Nguyên tắc vàng: "**có làm mới ghi, không làm dừng ghi**".
- Với mỗi NFR/BR bạn viết ra, phải chỉ được ra chỗ đã hiện thực trong code hoặc chỗ đã kiểm thử.

1.5. Nội dung trong tài liệu không khớp với phần mềm

- Có mục mô tả chức năng **không tồn tại trong phần mềm**
- Sơ đồ kiến trúc vẽ ra **không khớp với code trên repository**
- Tồn tại **hai bản Use Case Diagram khác nhau** trong cùng tài liệu
- Tài liệu mô tả một đăng, sản phẩm chạy một nẻo — "quá nhiều sự khác biệt với sản phẩm"
- **Mục tiêu ghi trong tài liệu rộng hơn nhiều so với thứ demo được** (tài liệu nói "thu thập và phân tích dữ liệu" nhưng sản phẩm chỉ thống kê được vài con số)
- **Thiếu hẳn một sơ đồ bắt buộc** (không có ERD, không có Use Case Diagram) — mất trắng điểm mục đó
- **Không mô tả giải thuật/thuật toán** đang dùng trong tài liệu
- Phần **Future Plan** hứa **những điều không khả thi** với nền tảng hiện tại (hứa "cải tiến thuật toán" trong khi nhóm chỉ dùng thư viện có sẵn, không hiểu bên trong)
- **Không nộp source code của một phần đã trình bày** trong tài liệu
- Không có thông tin về **doanh nghiệp/khách hàng thật** của đề tài; nếu là công ty giả định thì phải nói rõ và mô tả đủ nghiệp vụ
- **Nộp link thay vì nộp file** (sơ đồ, test case, thậm chí cả tài liệu chính) → người chấm không mở/đọc được nên **không đủ cơ sở để chấm phần tài liệu**. Luôn nhúng hình vào tài liệu và nộp file thật.

Cách tránh: Trước khi nộp, một thành viên đóng vai "reviewer lạ" đọc tài liệu và tự hỏi: *chức năng này có thật trong app không?*

1.6. Caption và tham chiếu chéo (cross-reference)

Đây là lỗi kỹ thuật trình bày rất hay bị bỏ qua nhưng luôn bị bắt.

Quy tắc	Đúng	Sai
Caption của Hình	Đặt bên dưới hình	Đặt bên trên
Caption của Bảng	Đặt bên trên bảng	Đặt bên dưới
Tham chiếu trong bài	"...như mô tả trong Hình 3.2 ", "...xem Bảng 4.1 "	Viết "hình dưới đây", "bảng sau"

Các lỗi cụ thể:

- Caption hình đặt trên hình, caption bảng đặt dưới bảng — **ngược quy ước**
- **Không có cross-reference:** hình và bảng chen vào nhưng không đoạn văn nào nhắc tới, người đọc không biết nhìn hình để làm gì
- Đánh số caption thủ công → sai thứ tự, trùng số, hoặc lệch khi thêm/bớt hình
- Caption chỉ ghi tên trống rỗng ("Hình 5") thay vì mô tả nội dung
- Danh mục hình / danh mục bảng không khớp với caption thật trong bài

Cách tránh:

- Dùng chức năng **Insert Caption** của Word cho mọi hình và bảng — số thứ tự tự động cập nhật.
- Dùng **Insert** → **Cross-reference** khi nhắc tới hình/bảng trong đoạn văn, không gõ tay số hiệu.
- Mỗi hình và bảng phải được **nhắc tới ít nhất một lần** trong phần nội dung trước khi xuất hiện.
- Sau khi hoàn tất, Ctrl+A → F9 để cập nhật toàn bộ số hiệu và danh mục.

2. Nhóm lỗi SƠ ĐỒ & MÔ HÌNH HÓA

Đây là nhóm lỗi bị bắt lỗi chi tiết nhất, vì sơ đồ được chấm đúng theo chuẩn ký hiệu.

2.1. Use Case Diagram

Lỗi	Ghi chú
Dùng sai include / extend	Lỗi phổ biến nhất trong toàn bộ nhóm lỗi sơ đồ
Vẽ sai quan hệ kế thừa (generalization)	
Sai cú pháp quan hệ, dùng mũi tên đen đặc	Quan hệ trong UC không dùng mũi tên tô đen
Use case thiếu actor (2 actor nhưng chỉ vẽ 1)	
Tên use case không phải động từ	Use case luôn phải là động từ + tân ngữ
Liên kết actor với use case chưa hợp lý	
Vẽ Use Case theo luồng xử lý	Use Case mô tả <i>chức năng</i> , không phải trình tự các bước
Sai nét nối giữa actor và use case	
Mô tả use case theo mẫu cũ	Phải dùng biểu mẫu mô tả use case mới nhất trên flm.fpt.edu.vn

2.2. Sequence Diagram

- **Thiếu số thứ tự** message — hoặc tệ hơn: **lúc có lúc không** (thiếu nhất quán)
- **Thiếu thanh activation bar**
- Thiếu dấu : trong tên object, thiếu schema
- **Thiếu các object tầng dưới:** controller, service, DAO/repository — vẽ từ frontend đi thẳng lên hub/backend là sai

- **Hộp alt không chia thành 2 lane** ứng với nhánh if/else, hoặc không đánh nhãn thành công/thất bại
- **Thiếu các luồng exception** (nếu nghiệp vụ có)
- **Gom nhiều use case vào 1 sequence** → không phù hợp
- Frontend phải chỉ **cụ thể đối tượng**, không vẽ chung chung
- Luồng phải đi theo đúng design pattern đã chọn
- Sequence không khớp với kiến trúc class đã mô tả ở phần thiết kế

2.3. ERD / Database Design

- Vẽ ở mức **entity** nhưng lại có **thuộc tính** → sai mức trừu tượng
- Nộp ERD **Conceptual** trong khi yêu cầu là **Physical**
- Vẽ theo Peter Chen thì **các con số (cardinality) phải nằm sát thực thể**
- **Tách ERD thành nhiều hình nhỏ** → nên gộp thành một sơ đồ tổng thể
- Đặt quan hệ sai bậc (ví dụ để n:n ở chỗ bản chất là 1:n)
- Đặt tên thực thể/quan hệ nhập nhằng
- Thiếu quan hệ giữa các thực thể → sinh ra tình huống vô lý về mặt dữ liệu
- Thiết lập kiểu dữ liệu / độ dài ký tự chưa hợp lý
- Lỗi chính tả, dư thực thể/thuộc tính không dùng đến
- **Ký hiệu không phân biệt được các bậc quan hệ** (0..n và 1..n vẽ giống hệt nhau)
- Không giải thích được quy ước của chính mình (nét liền / nét đứt / ký hiệu đầu mút nghĩa là gì)
- **Database Design phải là ảnh chụp từ CSDL thật**, không phải hình tự vẽ lý tưởng hóa
- Tài liệu nhắc tới một loại lưu trữ (NoSQL, cache...) nhưng **không hề mô tả** cách triển khai và dữ liệu lưu trong đó

2.4. System Architecture Diagram

Đây là sơ đồ bị hỏi xoáy nhiều nhất, vì nó được dùng để kiểm tra **bạn có thật sự hiểu hệ thống của mình không**.

- Vẽ **overview lẫn detail** trong cùng một hình → không hợp lý
- Đặt component sai chỗ (ví dụ: đặt ORM ra ngoài framework chứa nó)
- **Thiếu đường nối** tới third-party service
- Vẽ third-party chỉ nối với Backend trong khi thực tế Frontend cũng gọi
- Frontend nối trực tiếp tới File Storage/Database mà không giải thích được
- Vẽ có message queue / websocket / cache nhưng **không chỉ ra được chỗ nào implement**
- Kiến trúc **không nhất quán** giữa các phần của tài liệu
- Kiến trúc vẽ ra **không khớp với repository thật**

Cách tránh: Trước bảo vệ, mỗi thành viên phải làm được: *"Chỉ vào từng khối trong sơ đồ và mở đúng đoạn code tương ứng."* Đây là dạng yêu cầu rất hay xuất hiện.

2.5. Class Diagram & State Machine Diagram

Hai sơ đồ này hay bị làm qua loa hoặc bỏ quên, nhưng vẫn bị chấm.

- **Class Diagram:** thiếu hần trong tài liệu; hoặc **số lượng/kiểu liên kết giữa các class sai**
- **State Machine Diagram:** vẽ sai; **thiếu đối tượng gây ra việc chuyển trạng thái trên mỗi transition**
- Đặt State Machine Diagram ở **phụ lục** trong khi nó thuộc phần thiết kế chính

2.6. Các sơ đồ khác

- **Context Diagram:** không được gộp 2 entity; **nhãn trên luồng dữ liệu phải là danh từ, không phải động từ**; sai ký hiệu hình chữ nhật
- **Activity Diagram:** node quyết định phải là **hình thoi**, không phải ngũ giác
- **Product Overview Diagram:** sai ký hiệu hình chữ nhật bo góc
- **Mô hình phát triển phần mềm:** vẽ sai, hoặc thiếu phần diễn giải quy trình
- Sơ đồ không có chú thích, không có tiêu đề → người đọc không nhận ra đó là sơ đồ gì
- **Các sơ đồ không đồng bộ với nhau:** cùng một chức năng nhưng tên gọi, actor, luồng khác nhau ở mỗi sơ đồ

3. Nhóm lỗi PHÂN TÍCH NGHIỆP VỤ

Đây là nhóm lỗi nặng nhất về điểm số, vì nó cho thấy nhóm chưa hiểu bài toán.

3.1. Nghiệp vụ không sát thực tế

Các biểu hiện điển hình:

- Ràng buộc phi thực tế: một khách hàng chỉ được thuê một đơn vị tài nguyên; đối tượng đã tạo thì không bổ sung được thành phần; muốn đổi một thuộc tính thì phải xóa và tạo mới toàn bộ
- Quy trình bị **ngược so với thực tế**
- **Không khảo sát thực tế** ngữ cảnh triển khai, chỉ ngồi tự nghĩ tình huống
- Bỏ sót giai đoạn của quy trình thật (chỉ làm giai đoạn giữa, thiếu trước và sau)
- Cách tính giá/phí không theo cách ngành đó đang tính thật
- Bỏ qua thuế/VAT, hóa đơn điện tử khi bài toán liên quan tài chính
- Cộng dồn số liệu có **đơn vị tính khác nhau** → sai về bản chất; quy đổi đơn vị không tường minh
- Không xét các điều kiện môi trường thật (mất điện, mất mạng, địa hình đặc biệt...)
- **Dùng sai thuật ngữ chuyên ngành:** đặt tên một khái niệm nhưng bản chất lại là khái niệm khác → cho thấy nhóm chưa hiểu domain
- **Đặt tên trạng thái không đúng ngữ nghĩa** (ví dụ dùng "hoạt động / không hoạt động" trong khi bản chất là "cho phép / không cho phép")
- Giả định phi thực tế để né xử lý (mọi thiết bị luôn sẵn có, mọi khách hàng chỉ có một địa chỉ, mọi món hàng đều có mã định danh)
- Chức năng có nhưng **không phù hợp cách vận hành thật của ngành** (ví dụ cho rút tiền bất cứ lúc nào)
- **Tự phức tạp hóa bài toán** bằng những khái niệm trung gian không cần thiết
- **Đơn vị tính không thống nhất trong cùng một nghiệp vụ** (chỗ tính theo ngày, chỗ tính theo giờ)

- **Hiểu sai ngữ nghĩa của chính dữ liệu mình có** (lấy mốc "6 tháng gần nhất tính từ hôm nay" trong khi dữ liệu chỉ được thu thập tới ngày nào)
- **Định dạng dữ liệu pháp lý không chuẩn:** ngày tháng, số tiền, đơn vị tiền tệ trong hợp đồng/hóa đơn phải theo đúng quy ước
- Không có phương án cho **người dùng thiếu điều kiện tiêu chuẩn** (không có giấy tờ, không có điện thoại, làm thủ tục offline)

Cách tránh:

- **Đi hỏi người làm nghề thật** ít nhất một lần, ghi lại thành phụ lục khảo sát.
- Với mỗi quy trình, vẽ lại đủ 3 giai đoạn: *trước* – *trong* – *sau*.
- Tra cứu quy định/pháp lý hiện hành liên quan tới domain (thuế, bản quyền, dữ liệu cá nhân, luật chuyên ngành).

3.2. Thiếu ràng buộc và tình huống ngoại lệ

Câu hỏi "Nếu ... thì sao?" gần như chắc chắn sẽ được hỏi. Các lỗi hỏng thường gặp:

- Người tạo và người duyệt là **cùng một người**
- Đối tượng đã được duyệt rồi **vẫn sửa/edit được**
- Hủy hợp đồng/đơn hàng nhưng **không có bước phê duyệt việc hủy**
- Thanh toán một phần nhưng hệ thống ghi nhận "đã thanh toán xong"
- Không ràng buộc thứ tự xử lý trong quy trình
- Không xử lý khi thanh toán thất bại
- Không xử lý xung đột khi 2 người cùng thao tác trên một đối tượng
- Trạng thái "chờ" không có cơ chế thoát (chờ mãi khi chưa đủ điều kiện)
- Không có deadline hủy, hoặc deadline của 2 dịch vụ liên quan không khớp nhau
- Phân quyền sai: thu hồi quyền của vai trò này làm khóa luôn vai trò khác
- Không ràng buộc dữ liệu → sinh ra giá trị vô lý (ngày mặc định 1/1/1, số tiền âm)
- Không kiểm soát sự phù hợp giữa người dùng và tài nguyên họ tạo ra
- **Không xử lý khi hệ thống/thiết bị chết giữa chừng** (mất kết nối đúng lúc check-in/check-out, giao dịch dở dang)
- Quy trình phụ thuộc vào một người duyệt thủ công mà **không có cơ chế nhắc/timeout** nếu người đó quên
- **Không có khả năng truy vết** (audit log) khi cần điều tra sự cố trong một khoảng thời gian
- Thao tác vẫn thực hiện được ở thời điểm lỡ ra bị chặn (sửa lịch của ngày hiện tại trong khi quy định chỉ cho sửa từ ngày mai)

Cách tránh — bảng "Nếu... thì sao?":

Với **mỗi** mainflow, viết ra tối thiểu 5 câu hỏi dạng này và trả lời bằng code:

1. Nếu thanh toán fail giữa chừng thì sao?
2. Nếu 2 người thao tác cùng lúc thì sao?
3. Nếu người dùng hủy sau khi đã được duyệt thì sao?
4. Nếu dữ liệu vượt giới hạn hợp lý (âm, quá lớn, quá xa) thì sao?
5. Nếu vai trò A cố tình làm việc của vai trò B thì sao?

3.3. Con số trên UI không giải thích được

Một dạng kiểm tra rất hay gặp là chỉ vào **một con số cụ thể trên màn hình** và yêu cầu giải thích. Các lỗi thường gặp:

- Không giải thích được công thức tính phí vận chuyển / phí dịch vụ
- Lợi nhuận tính sai vì bỏ sót một khoản khấu trừ (hoa hồng, thuế...)
- Tổng doanh thu không có lịch sử giao dịch để đối chiếu
- Không phân biệt được tiền vào / tiền ra trên UI
- Xuất hóa đơn ra **số âm**
- Số liệu thống kê không lọc được theo phạm vi (dự án, chiến dịch, chi nhánh)
- Dòng tiền vào minh bạch nhưng **dòng tiền ra thì không**
- **Không ghi log doanh thu / phần trích lại của nền tảng** → không đối chiếu được
- Số dư khả dụng không khớp khi thực hiện rút tiền
- Thời gian ghi nhận giao dịch sai
- Trọng số/tỷ lệ tính điểm áp dụng như nhau cho các tập dữ liệu có quy mô khác nhau → kết quả không có ý nghĩa
- Không giải thích được **công thức tính lương, hoa hồng, KPI**

Cách tránh: Chuẩn bị sẵn một file ghi rõ **mọi công thức tính tiền/điểm** trong hệ thống kèm ví dụ số cụ thể, và luyện tập giải thích bất kỳ con số nào xuất hiện trên demo.

3.4. Business Rule viết cho có

Business Rule là phần bị soi kỹ và rất hay bị chê. Các lỗi thường gặp:

- **Số lượng BR quá ít so với quy mô ứng dụng** — vài chục rule cho một hệ thống lớn bị đánh giá là quá sơ sài
- **BR viết ra nhưng code không hề kiểm tra** (BR ghi "tối thiểu 50 câu hỏi" nhưng hệ thống không validate)
- **BR mô tả sai so với thứ đã hiện thực** — người chấm đọc BR rồi thử ngay trên app và phát hiện lệch
- **Thiếu BR cho những ràng buộc quan trọng nhất:** giới hạn số người, điều kiện hủy, quy tắc trích phần trăm, quy tắc tính giá
- BR không ghi nhận các **tham số cấu hình** đang tồn tại trong hệ thống
- BR viết bằng AI, nội dung chung chung, không đúng với nghiệp vụ thật

Cách tránh: Sau khi code xong, rà ngược từ code ra BR — mọi câu lệnh if mang tính nghiệp vụ đều phải có một BR tương ứng, và ngược lại.

4. Nhóm lỗi HARDCODE & CẤU HÌNH

Đây là lỗi bị bắt gần như trong mọi buổi bảo vệ, và là một trong những lý do chính dẫn tới kết quả không đạt.

Những thứ hay bị phát hiện hardcoded:

- Phần trăm hoa hồng, phí nền tảng, tỷ lệ trích giữ lại

- Ngưỡng nghiệp vụ (số tiêu chí cần đạt, tỷ lệ tối thiểu để được hoàn tiền...)
- Ngưỡng tin cậy của model AI
- Giá gói dịch vụ, giới hạn khoảng cách, trọng số thuật toán
- Nội dung email/thông báo
- Danh mục cấu hình (loại đề, số vòng của quy trình...)
- Thông tin thiết bị IoT khai báo từng cái một
- **Mã OTP để cố định một giá trị trong code** — vừa là hardcoded vừa là lỗ hổng bảo mật
- Thậm chí cả kết quả trả về của module AI

Câu hỏi kinh điển: "Giờ tôi muốn đổi con số này từ 70% lên 80% thì làm thế nào? Hãy demo ngay."

Cách tránh:

- Mọi tham số nghiệp vụ phải nằm trong **bảng cấu hình trong DB**, có màn hình admin để sửa, và **sửa xong có hiệu lực ngay**.
- Sửa file JSON/appsettings rồi restart **vẫn bị coi là hardcoded**.
- Chuẩn bị sẵn một màn hình "System Configuration" và chủ động demo nó — điều này gây ấn tượng rất tốt.
- Rà lại code, tìm mọi **magic number** và đưa vào config.

5. Nhóm lỗi liên quan đến AI trong đề tài

Rất nhiều đề tài có chữ "AI", "thông minh", "smart" trong tên. Phần AI luôn bị kiểm tra kỹ xem có thật sự tạo giá trị không.

Các lỗi thường gặp:

- Tên đề tài có AI nhưng khi demo **phần AI mờ nhạt**, bài toán quy về CRUD/đặt lịch thông thường
- Gọi là AI nhưng thực chất chỉ là **if-else / rule-based**
- Dùng model có sẵn, **không training**, không chỉ ra được đóng góp riêng của nhóm
- Model chỉ nhận diện đúng với **đúng một mẫu dữ liệu đã chuẩn bị sẵn**, mẫu khác cho kết quả sai
- Độ chính xác quá thấp nhưng vẫn báo cáo là đạt
- Không trả lời được **model được lưu ở đâu, phiên bản nào**
- Không có **metric đánh giá** chất lượng đầu ra
- Không có **actor duyệt lại** kết quả do AI sinh ra → không đảm bảo độ tin cậy
- Chatbot trả lời sai vì dùng dữ liệu cũ, không cập nhật theo quy định hiện hành
- Không quản lý giới hạn token / chi phí AI của người dùng
- Xử lý dữ liệu làm mất thông tin (chuyển speech → text rồi mới phân tích, làm mất ngữ điệu)
- AI sinh nội dung nhưng **không ràng buộc tính phù hợp** (độ tuổi, ngữ cảnh sử dụng)
- AI hỗ trợ quá nhiều làm **vai trò của con người trong hệ thống trở nên mờ nhạt**

- **Không giải thích được ngưỡng tin cậy** đang dùng: con số đó từ đâu ra, dựa trên thí nghiệm nào
- **Chiến lược training không hợp lý** về mặt kiến trúc (train riêng cho từng đơn vị/từng điểm sử dụng trong khi có thể dùng chung một model)
- Gán nhãn/gom nhóm dữ liệu sai bản chất → model trả về kết quả không liên quan
- **Lạm dụng từ "smart" / "thông minh"** trong tên đề tài và tài liệu mà không có gì tương xứng
- Đưa AI vào chỗ mà một truy vấn hoặc một công thức đơn giản đã giải quyết được tốt hơn

Cách tránh:

- Trả lời rõ được 4 câu: *AI giải quyết vấn đề gì? Dữ liệu ở đâu? Đo bằng metric nào, kết quả bao nhiêu? Ai kiểm chứng đầu ra?*
- Chuẩn bị **nhiều mẫu dữ liệu khác nhau** để demo, kể cả mẫu khó — người chấm thường sẽ tự lấy mẫu mới để thử.
- Nếu là rule-based thì nói thẳng là rule-based, đừng gọi là AI.

6. Nhóm lỗi KIỂM THỬ (Test)

- **Test Report còn nhiều case Fail** mà vẫn nộp, không giải thích
- Số liệu tổng hợp trong test report **sai và không nhất quán**
- Test report **chưa nhập đầy đủ các trường thông tin**
- Test case viết như **hướng dẫn sử dụng** thay vì test case
- Cách viết **không phải system test**, số lượng test case nhiều nhưng không có giá trị
- **Bước (STEP) không đủ chi tiết** để người khác tái hiện được từ góc độ user
- Unit Test **sai, không tách theo từng tham số**, không ghi đủ thông tin người thực hiện
- Test chỉ phủ được một phần luồng (chỉ vài bước đầu của quy trình nhiều bước)
- Các test case **không liên kết với nhau** thành kịch bản có nghĩa

Cách tránh:

- Test case phải **trace ngược về requirement**: mỗi FR có ít nhất 1 test case.
- Mỗi step phải viết ở mức người lạ đọc vào làm lại được.
- Nếu còn case Fail: hoặc sửa, hoặc ghi rõ **known issue + lý do + kế hoạch xử lý**. Đừng giấu.

7. Nhóm lỗi khi DEMO

Đây là nơi mất điểm nhanh nhất, vì người chấm nhìn thấy trực tiếp.

7.1. Các thất bại điển hình

- Chức năng lỗi khi demo, demo lại **vẫn lỗi**
- **Hết thời gian quy định vẫn chưa demo được các luồng chính**
- **Không có dữ liệu để demo tiếp** → buổi bảo vệ bị dừng giữa chừng. Đây là kịch bản xấu nhất và đã thực sự xảy ra.

- **Thiết bị phần cứng (IoT) hỏng ngay hôm bảo vệ** → không demo được các luồng liên quan → không đạt
- Gặp lỗi UI nhưng **lướt qua, bấm next**, không giải thích → bị đánh giá thiếu trung thực
- **Không chuẩn bị dữ liệu demo trước**, phải ngồi nhập liệu tại chỗ; dữ liệu giả lập không hợp lý, không giống thực tế
- Kịch bản demo lộn xộn, trình bày không theo mạch (nhảy giữa các nghiệp vụ không liên quan), gây khó theo dõi
- Có chức năng đã làm nhưng **quên không demo**, hoặc demo thiếu đúng phần quan trọng nhất của đề tài
- Setup hệ thống dễ dãi "cho tiện demo" (bỏ ràng buộc) rồi bị phát hiện
- **Dùng chung một tài khoản/email cho nhiều vai trò khác nhau** khi demo → không chứng minh được hệ thống phân quyền đúng
- Trình bày xong toàn bộ lý thuyết rồi mới demo tất cả — nên **trình bày tới luồng nào thì demo ngay luồng đó**
- Không giải thích được nguyên nhân lỗi, chỉ biết "đổi thiết bị khác thì hết"

7.2. Nguyên tắc demo

1. **Chuẩn bị dữ liệu mẫu đầy đủ, thật, đa dạng** — có lịch sử giao dịch, nhiều user, nhiều trạng thái.
2. **Viết kịch bản demo** theo mainflow, phân vai rõ ai demo phần nào, tập chạy thử ít nhất 2 lần có bấm giờ.
3. **Không bao giờ lướt qua lỗi**. Dừng lại, nói rõ: *"Đây là lỗi X, nguyên nhân là Y, nhóm đã ghi nhận"*. Sự trung thực được đánh giá cao; né tránh bị trừ điểm nặng.
4. **Đừng sửa hệ thống cho "dễ demo"**. Nếu cần demo tình huống phụ thuộc thời gian, hãy chỉnh giờ server thay vì gỡ bỏ ràng buộc.
5. Chuẩn bị sẵn **phương án dự phòng** (video ghi màn hình mainflow) cho tình huống mạng/thiết bị hỏng. Nếu đề tài có phần cứng: **mang theo bộ thiết bị thứ hai** và test lại ngay trước giờ bảo vệ.
6. Chuẩn bị sẵn **IDE đã mở repository**, vì rất hay bị yêu cầu mở code ngay tại chỗ.
7. Đặt **mật khẩu đơn giản** cho các tài khoản demo — gõ mật khẩu dài và sai nhiều lần ngay tại buổi bảo vệ làm mất thời gian và mất điểm trình bày.

7.3. Slide thuyết trình

- Slide phải có phần **tổng kết những gì đạt được và những hạn chế còn tồn tại** — thiếu phần này bị nhắc riêng.
- Slide cần chẵn chu, đúng thuật ngữ tiếng Anh, phù hợp với chủ đề đề tài.
- Giới thiệu rõ **các thiết bị/thành phần đặc thù** của hệ thống: dùng để làm gì, tương tác thế nào.
- Trình bày theo mạch người nghe hiểu được, không nhảy cóc giữa các nghiệp vụ.

8. Nhóm lỗi khi TRẢ LỜI CÂU HỎI

8.1. Không hiểu code của chính mình

Tình huống rất xấu và khá phổ biến: được yêu cầu mở source code một chức năng, sinh viên trả lời *"em dùng thư viện gì em không nhớ"*, hoặc thừa nhận đã **dùng AI sinh code nhưng không hiểu nguyên lý**.

Các biến thể khác:

- "Phần đó bạn khác làm nhưng bạn ấy nghĩ rồi"
- Không chỉ ra được chỗ nào trong code xử lý một nghiệp vụ cụ thể
- Không giải thích được cơ chế authentication/authorization trên code
- Không trả lời được service call bị fail thì được handle ở đâu

Cách tránh:

- **Mọi thành viên phải đọc và hiểu được toàn bộ mainflow**, không chỉ phần mình làm.
- Trước bảo vệ, tổ chức một buổi "hỏi chéo": mỗi người bị nhóm hỏi 10 câu về phần người khác làm.
- Nếu có dùng AI hỗ trợ code: bắt buộc phải đọc lại, hiểu, và giải thích được từng đoạn.

8.2. Trả lời không rõ ràng / vòng vo

Những nhận xét như *"trả lời không rõ ràng"*, *"chưa trả lời được"*, *"giải thích chưa hợp lý"*, *"chưa nắm vững vấn đề kỹ thuật"* luôn đi kèm điểm thấp.

Cách tránh:

- Trả lời theo cấu trúc: **Có/Không → Lý do → Chỗ hiện thực → Hạn chế nếu có**.
- Nếu chưa làm, nói thẳng: *"Nhóm chưa xử lý tình huống này"*. Ngắn gọn và trung thực tốt hơn vòng vo.
- Không tranh cãi. Ghi nhận, và nếu có căn cứ thì trình bày ngắn gọn.

8.3. Không nắm được định vị của sản phẩm

Những câu tưởng đơn giản mà nhiều nhóm trả lời sai:

- *"Đây là hệ thống hay nền tảng?"* (khác nhau ở chỗ ai là người cấu hình: admin hay chủ tài nguyên)
- *"Sản phẩm này khác gì với các sản phẩm đã có trên thị trường?"*
- *"Đề tài này có gì đặc sắc, tại sao cần một hệ thống riêng?"*
- *"Nhóm focus vào ứng dụng hay vào AI?"*

Cách tránh: Chuẩn bị một **elevator pitch 60 giây** trả lời: bài toán – ai dùng – khác biệt so với đối thủ – giá trị cốt lõi.

9. Nhóm lỗi QUY TRÌNH & LÀM VIỆC NHÓM

Đây là nhóm lỗi dẫn tới hậu quả nghiêm trọng nhất.

Lỗi	Hậu quả
Không được GVHD duyệt ra bảo vệ	Không đạt
Nhóm không có mặt đúng giờ bảo vệ	Không đạt
Thành viên vắng các buổi review, đóng góp rất ít	Cá nhân đó không đạt và phải làm lại đề án , dù cả nhóm đạt
Không cải thiện theo góp ý của các lần Review đánh giá	Không đạt hoặc phải bảo vệ lần 2
Thành viên bỏ nhóm giữa chừng, phần việc bỏ trống	Không giải thích được khi bị hỏi tới phần đó
Git tổ chức lộn xộn, không phân biệt được ai làm phần nào	Không có căn cứ chứng minh đóng góp cá nhân

Cách tránh:

- **Nhóm phải tự tổ chức buổi tổng duyệt.** Lập bảng theo dõi từng góp ý của các lần Review và góp ý của GVHD → trạng thái xử lý → bằng chứng đã sửa. Mang bảng này đi bảo vệ.
- Ghi nhận đóng góp qua **commit history + biên bản họp nhóm hàng tuần.**
- Xác nhận với GVHD về việc duyệt ra bảo vệ **trước hạn ít nhất 2 tuần.**

10. Nhóm lỗi về DỮ LIỆU, BẢO MẬT & PHÁP LÝ

Nhóm lỗi này ít được sinh viên chuẩn bị nhưng lại bị hỏi khá thường xuyên.

- **Thu thập thông tin quá mức cần thiết** về người dùng (số điện thoại, địa chỉ, CCCD khi không cần)
- **Rò rỉ thông tin:** nhập một trường định danh thì hệ thống hiển thị luôn thông tin cá nhân → có thể dò thông tin người khác
- Yêu cầu xác thực giấy tờ tùy thân ở nơi không hợp lý
- Xác minh giấy tờ/mã số doanh nghiệp **thủ công bằng mắt**, không có cơ chế đối chiếu
- Không nghiên cứu **policy của nền tảng bên thứ ba** khi đề tài phụ thuộc vào nó
- Không xử lý **bản quyền** nội dung người dùng tải lên, thiếu cam kết của chủ sở hữu nội dung
- Cập nhật quy định pháp luật **quá trễ** so với thực tế
- **Không ghi rõ nguồn dữ liệu** lấy từ bên thứ ba hoặc cơ quan nhà nước
- Không kiểm soát **tính phù hợp nội dung với lứa tuổi** khi người dùng là trẻ em
- NFR ghi "dữ liệu người dùng phải bảo mật" nhưng **chưa hiện thực**
- API key / secret để lộ trong source code
- **Lộ đường dẫn trực tiếp tới tài nguyên** (link file, video, tài liệu) → ai có link đều tải được, mọi cơ chế phân quyền phía trên trở nên vô nghĩa
- Bảo vệ nội dung **chỉ ở phía frontend** (chặn phím tắt, chặn chuột phải) và chỉ hoạt động trên vài trình duyệt → không phải là bảo mật

- Chuyển tiền cho người bán ngay sau giao dịch → rủi ro khi có khiếu nại

Cách tránh:

- Áp dụng nguyên tắc **thu thập tối thiểu**: chỉ lấy dữ liệu thực sự dùng đến.
- Có mục **Legal & Compliance** gắn trong tài liệu: bản quyền, dữ liệu cá nhân, quy định ngành.
- Quét repository tìm secret/API key trước khi nộp.
- Ghi nguồn cho mọi dữ liệu bên thứ ba.

11. Nhóm lỗi về UI/UX

- UI dày đặc, chưa gọn gàng — hoặc ngược lại: **trống, đơn điệu**
- Nút "Duyệt" và "Không duyệt" đặt **quá gần nhau**
- Giao diện **dư thừa** (cùng một điều khiển xuất hiện ở hai nơi)
- **Không thống nhất ngôn ngữ** trên giao diện
- Nhấn sai nghĩa so với chức năng thật của nút
- Biểu đồ thống kê **không mang nhiều ý nghĩa**, khó hiểu
- Thiếu thông tin quan trọng trên màn hình (ngày kết thúc, số lượng, đơn vị cung cấp, số bản ghi bị từ chối)
- Nhập liệu quá nhiều, quy trình thủ công trong khi có thể tự động
- Chọn sai định dạng xuất báo cáo (PDF trong khi Excel phù hợp hơn)
- Giao diện chưa phù hợp đối tượng người dùng (trẻ em cần màu sắc, hình ảnh sinh động)
- Dùng thuật ngữ kỹ thuật khó hiểu với người dùng cuối, nên thay bằng từ ngữ đời thường
- **Danh sách dài không có phân trang**, không có tìm kiếm/lọc → không dùng được với dữ liệu thật
- **Bố cục không cân đối**, chi tiết hóa những chỗ không cần thiết mà bỏ trống chỗ quan trọng
- **Luồng thao tác trên mobile quá nhiều bước** cho một tác vụ đơn giản
- Định dạng hiển thị số tiền, ngày tháng không theo quy ước chung

12. CHECKLIST TỰ KIỂM TRƯỚC BẢO VỆ

D-14 (hai tuần trước)

- ☐ Tải template chính thức mới nhất từ FLM, đối chiếu toàn bộ tài liệu
- ☐ Lập bảng: **mọi góp ý của các lần Review và của GVHD → đã xử lý chưa → bằng chứng**
- ☐ Rà soát toàn bộ magic number trong code → đưa vào config có UI quản trị
- ☐ Xác nhận GVHD đồng ý duyệt ra bảo vệ
- ☐ Kiểm tra mọi thành viên đều có commit và phần việc rõ ràng

D-7 (một tuần trước)

- ☐ Kiểm tra lại **8 loại sơ đồ** theo mục 2 (Use Case, Sequence, Class, State Machine, ERD, Architecture, Context, Activity)
- ☐ Kiểm tra các sơ đồ **đồng bộ với nhau**: cùng tên chức năng, cùng actor, cùng luồng
- ☐ Đối chiếu sơ đồ kiến trúc với repository thật — từng khối phải trỏ được tới code
- ☐ Rà lại toàn bộ FR/NFR/BR: **có làm mới ghi**; mọi if nghiệp vụ trong code đều có BR tương ứng
- ☐ Hoàn thiện Test Report: không còn case Fail chưa giải thích, số liệu nhất quán, dùng đúng template FLM
- ☐ Kiểm tra danh mục tài liệu: Acknowledgement (có tên GVHD), User Guide, Class Diagram, danh mục bảng/hình, bảng viết tắt, số trang, mục lục
- ☐ Xóa hết placeholder/tag còn sót của template; phóng to mọi sơ đồ tới mức đọc được
- ☐ **Caption: hình ở dưới, bảng ở trên**; mọi hình/bảng đều được nhắc tới bằng cross-reference trong nội dung
- ☐ Cập nhật toàn bộ trường tự động (Ctrl+A → F9): mục lục, danh mục hình/bảng, số hiệu caption
- ☐ Kiểm tra danh mục tài liệu tham khảo: nguồn đáng tin cậy, thông tin còn hiệu lực, đủ thông tin trích dẫn
- ☐ Lập bảng công thức tính toán (tiền, điểm, lương, hoa hồng, trọng số) kèm ví dụ số cụ thể
- ☐ Chuẩn bị dữ liệu demo phong phú, nhiều trạng thái, có lịch sử

D-3

- ☐ Viết kịch bản demo, phân vai, chạy thử **có bấm giờ** ít nhất 2 lần
- ☐ Buổi "hỏi chéo": mỗi thành viên trả lời 10 câu về phần người khác làm
- ☐ Chuẩn bị trả lời "Nếu... thì sao?" cho mỗi mainflow (tối thiểu 5 tình huống)
- ☐ Chuẩn bị elevator pitch 60 giây về định vị sản phẩm
- ☐ Quét repository tìm API key / secret bị lộ
- ☐ Đọc lại tài liệu ở chế độ print preview để bắt lỗi trình bày

D-1

- ☐ Ghi màn hình video mainflow làm phương án dự phòng
- ☐ Chuẩn bị máy dự phòng, kiểm tra mạng; **nếu có phần cứng: test lại và mang bộ dự phòng**
- ☐ Kiểm tra lại dữ liệu demo còn nguyên vẹn, đủ để chạy hết kịch bản
- ☐ Đặt mật khẩu đơn giản cho tài khoản demo, ghi ra giấy
- ☐ Mở sẵn: app đã deploy, IDE với repository, tài liệu, file công thức, bảng theo dõi góp ý Review
- ☐ Xác nhận lại giờ và địa điểm bảo vệ

Ngày bảo vệ

- ☐ Có mặt sớm ít nhất 30 phút
 - ☐ Demo đúng kịch bản, **không lướt qua lỗi**
 - ☐ Trả lời: Có/Không → Lý do → Chỗ hiện thực → Hạn chế
 - ☐ Ghi chép lại toàn bộ góp ý để chỉnh sửa và nộp lại đúng hạn
-

13. NHỮNG CÂU HỎI HAY GẶP NHẤT

Chuẩn bị sẵn câu trả lời cho những câu dưới đây — chúng lặp lại ở hầu hết các buổi bảo vệ:

Về cấu hình:

1. Con số này (% , ngưỡng, phí) có thay đổi được không? Demo ngay cho tôi xem.
2. Ai là người cấu hình cái này — admin hay chủ tài nguyên?

Về kiến trúc & code:

3. Chỉ vào sơ đồ kiến trúc và mở đoạn code tương ứng.
4. Thành phần X trong sơ đồ dùng để làm gì? Tại sao chọn công nghệ này mà không phải cái kia?
5. Service này đồng bộ hay bất đồng bộ? Call fail thì handle ở đâu?

Về nghiệp vụ:

6. Nếu người dùng hủy sau khi đã duyệt thì sao?
7. Nếu thanh toán thất bại giữa chừng thì sao?
8. Người tạo và người duyệt có thể là cùng một người không?
9. Nghiệp vụ này ngoài thực tế có đúng như vậy không? Nhóm đã khảo sát chưa?

Về con số:

10. Giải thích con số này trên màn hình được tính như thế nào?
11. Cho xem dòng tiền vào và dòng tiền ra.

Về AI:

12. AI trong hệ thống làm gì? Có training không? Đánh giá chất lượng bằng gì?
13. Ai kiểm chứng kết quả AI trả về?

Về Business Rule:

14. Ràng buộc này nhóm ghi trong Business Rule — chỉ cho tôi chỗ code kiểm tra nó.
15. Vì sao hệ thống lớn thế này mà chỉ có ngần này Business Rule?

Về tài liệu:

16. Mở template chính thức của trường ra, đối chiếu với tài liệu của nhóm.
17. Chức năng ghi trong tài liệu này có trong phần mềm không?
18. Sơ đồ này vẽ tay hay chụp từ hệ thống thật?
19. Hình/bảng này được nhắc tới ở đoạn nào trong bài?

20. Thuật toán nhóm dùng là gì? Mô tả ở mục nào trong tài liệu?

14. BẢNG TỰ ĐÁNH GIÁ RỦI RO

Cho điểm mỗi dòng: **0 = chưa làm, 1 = làm một phần, 2 = làm tốt**

#	Tiêu chí	Điểm
1	Tài liệu đúng template chính thức, đủ mọi trang bắt buộc	
2	8 loại sơ đồ đúng chuẩn ký hiệu, nhất quán với nhau và với code	
3	Không còn tham số nghiệp vụ nào hardcoded	
4	Mọi mainflow đã xử lý ít nhất 5 tình huống ngoại lệ	
5	Giải thích được mọi con số hiển thị trên UI	
6	Nghiệp vụ đã được khảo sát/kiểm chứng với người làm nghề thật	
7	Test Report đầy đủ, số liệu nhất quán, không còn Fail chưa giải thích	
8	Mọi thành viên hiểu và giải thích được toàn bộ mainflow + code	
9	Đã xử lý hết góp ý của các lần Review và của GVHD, có bằng chứng	
10	Kịch bản demo đã chạy thử có bấm giờ, có dữ liệu mẫu đầy đủ	
11	AI (nếu có) có metric đánh giá và cơ chế con người kiểm chứng	
12	Đã rà soát dữ liệu cá nhân, bản quyền, secret key, đường dẫn tài nguyên	
13	Business Rule đầy đủ và được code kiểm tra thật , không chỉ nằm trên giấy	
14	Có phương án dự phòng cho dữ liệu demo và phần cứng (nếu có)	
15	Caption đúng quy ước và mọi hình/bảng có cross-reference trong nội dung	

Cách đọc kết quả:

- **26 – 30 điểm:** Sẵn sàng bảo vệ.
- **18 – 25 điểm:** Có khả năng rơi vào kết quả "đạt nhưng phải chỉnh sửa và nộp lại tài liệu". Tập trung vào các dòng đang 0-1.
- **Dưới 18 điểm:** Rủi ro cao phải bảo vệ lần 2. Cần ưu tiên ngay các mục 1, 3, 4, 9, 10, 14.

15. Ba lời khuyên quan trọng nhất

1. **Trung thực luôn được điểm cao hơn che giấu.** Các nhóm chủ động nói ra hạn chế của mình được đánh giá tốt; những nhóm lướt qua lỗi hoặc trả lời vòng vo bị trừ điểm nặng.

2. **Các buổi Review là cơ hội sửa sai, không phải thủ tục.** "Không tiếp thu góp ý của các lần Review đánh giá" là một trong những lý do trực tiếp dẫn tới kết quả không đạt.
3. **Tài liệu là phần dễ ăn điểm nhất và bị mất điểm nhiều nhất.** Sản phẩm chạy tốt nhưng tài liệu sai sơ đồ, thiếu trang, sai template vẫn khiến nhóm phải làm lại và nộp lại. Đây là công việc không cần sáng tạo, chỉ cần cẩn thận.